PLAYBOOK DE PUBLICAÇÃO · V2

Cartilha de deploy

Passo a passo para publicar o **Diretório de Contatos v2** no Render com login Google, validar o que ficou no ar e saber como voltar atrás. Escrita para ser seguida com o painel aberto ao lado, sem precisar lembrar de nada.

Projeto Diretório de Contatos v2	Repositório EnzoMoulin/diretorio-contatos-v2
Stack Python 3.15 (stdlib) + SQLite + HTML/CSS/JS	Hospedagem Render — web service, plano free
Login Google OAuth 2.0 (somente e-mail e perfil)	Versão da cartilha 08/11/2026

00 Antes de começar

5 MIN DE LEITURA

O deploy tem três partes que dependem uma da outra: o serviço no Render, o cliente OAuth no Google Cloud e as variáveis de ambiente que ligam os dois. Se qualquer uma faltar, a página pública abre mas ninguém consegue entrar no admin — é o modo de falha mais comum desta etapa.

PEÇA	PARA QUE SERVE	SE FALTAR
Serviço no Render	Executa `app.py` e serve a página pública e o admin	Não há o que acessar
Cliente OAuth (Google)	Permite entrar com conta Google e receber o e-mail verificado	O botão de entrar falha com `redirect_uri_mismatch`
`ADMIN_EMAILS`	Define quem é administrador	O sistema abre, mas ninguém administra
`APP_BASE_URL`	Monta links de e-mail e o retorno do login	Links apontam para `localhost`

O que já está pronto: 27 testes automatizados passando, integração contínua verde no GitHub Actions, `render.yaml` versionado, banco mascarado para demonstração (`seed_mascarado.json`) e login local desligado em produção (`LOCAL_LOGIN=0`). Falta apenas a configuração de painel descrita aqui.

01 O que o deploy vai criar

Página pública

`/` — consulta de comarcas, sede, telefones úteis e favoritos, sem login.

Admin

`/admin` — só depois do login Google; gestão, solicitações, notificações e usuários.

Banco

`contatos.db` criado no primeiro boot e preenchido pelo seed mascarado.

Endereço

`https://diretorio-contatos-v2.onrender.com`, se o nome estiver livre.

Atenção ao disco: o plano free não tem disco persistente. Cada novo deploy ou reinício recria o `contatos.db` a partir do seed — o que mantém o demo sempre limpo. Como o ambiente é somente leitura, não há edição de visitante a perder; o que não sobrevive ao reinício são contas, pedidos e aprovações. Isso é esperado nesta vitrine, não um defeito.

02 Decisões que precisam estar tomadas

ANTES DO CLIQUE

Três escolhas mudam o comportamento do ambiente publicado. Elas já estão refletidas em `render.yaml`; confirme se continuam valendo antes de aplicar.

A Quem é administrador

`ADMIN_EMAILS` é uma lista separada por vírgula. Só quem está nela recebe papel de administrador — e a promoção é aplicada no login. Sem essa variável, o sistema funciona com todos os usuários comuns e nenhum admin.

Valor sugerido: `seu@gmail.com`

B Aprovação de acesso — decisão adotada: `0`

O demo público sobe com `SIMULACAO_AUTO_APROVAR=0`: é **somente leitura** para quem visita. Um pedido de acesso fica pendente e só o dono aprova, em `/admin → Solicitações`, exatamente como em produção.

Por quê: com `1` qualquer conta Google que pedisse acesso recebia permissão de edição, e o demo ficava vulnerável a alteração dos dados por terceiros — quem visita não precisa disso para avaliar o sistema, e a edição continua documentada no README e nas capturas do painel.

Onde o `1` continua valendo: execução local e os testes automatizados, onde a aprovação automática é sinalizada na tela; é o modo que a suíte assume.

C Envio de e-mail

O código de aprovação e recusa já chama o envio, mas as variáveis `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` e `SMTP_FROM` ainda não estão definidas. Sem elas, nada é enviado e o pedido continua registrado no painel.

Decisão: publicar sem e-mail (padrão) ou configurar o SMTP neste mesmo passo.

03 Deploy do serviço no Render

≈ 8 MIN

1 Criar o serviço por Blueprint

No painel do Render: **New → Blueprint**, escolher o repositório `EnzoMoulin/diretorio-contatos-v2` e confirmar. O Render lê o `render.yaml` do repositório e propõe o serviço já configurado.

2 Preencher o que o arquivo não guarda

O `render.yaml` marca `ADMIN_EMAILS` como `sync: false`: o painel pede o valor na hora de aplicar. Informe o e-mail definido em 2A. Se preferir criar o serviço manualmente (**New** → **Web Service**), use exatamente:

- **Runtime:** Python 3 · **Build command:** `echo ok`
- **Start command:** `HOST=0.0.0.0 python3 app.py`
- **Health check path:** `/healthz` · **Plano:** Free

3 Conferir as variáveis do arquivo

Devem estar presentes, com estes valores: `PYTHON_VERSION=3.15.2` (release estável atual; o Render aceita qualquer versão lançada a partir da 3.7.3), `INTRANET_ONLY=0`, `LOCAL_LOGIN=0` e `SIMULACAO_AUTO_APROVAR=0` (decisão 2B: demo somente leitura).

4 Primeiro deploy

Aplicar e acompanhar o log até `Live`. O build é imediato porque não há dependências — todo o backend usa a biblioteca padrão do Python. Depois do primeiro boot, abrir a URL e confirmar que a página pública responde.

5 Fixar a URL pública no app

Em **Environment**, adicionar `APP_BASE_URL` com a URL real do serviço (sem barra no fim). Isso corrige os links de e-mail e o retorno do login.

04 Login Google (OAuth) ≈ 12 MIN

1 Criar o cliente OAuth

No Google Cloud Console, no mesmo projeto usado antes: **APIs e serviços** → **Credenciais** → **Criar credenciais** → **ID do cliente OAuth**. Tipo de aplicativo: **Aplicativo da Web**.

2 Registrar as origens e o retorno

Estes dois campos precisam bater com a URL do Render, caractere por caractere:

- **Origens JavaScript autorizadas:** `https://diretorio-contatos-v2.onrender.com`
- **URIs de redirecionamento autorizados:** `https://diretorio-contatos-v2.onrender.com/oauth/callback`

O caminho de retorno é `/oauth/callback` — sem ele, o Google recusa com `redirect_uri_mismatch`.

3 Colar o JSON no Render

Baixar o JSON do cliente e colar o conteúdo inteiro em `GOOGLE_CLIENT_JSON` (Environment do Render, marcado como segredo). Nada de subir esse arquivo para o repositório.

4 Tela de consentimento — pode ficar em Teste

Com o demo somente leitura, o visitante não precisa entrar: a página pública `/` abre sem login. Mantendo a tela em **Teste**, só as contas cadastradas como testadores conseguem fazer login — na prática, só o dono do ambiente. Publique (**Em produção**) apenas se quiser que terceiros entrem, o que só é necessário se a aprovação automática for ligada.

Os escopos usados (`openid`, `userinfo.email`, `userinfo.profile`) são não sensíveis: a publicação não exige verificação do Google.

> **Depois de qualquer mudança nessas variáveis,** o Render reinicia o serviço sozinho. Aguarde o `Live` antes de testar o login — durante o reinício o callback falha.

05 Validação pós-deploy

CHECKLIST

Faça na ordem. Cada item tem um resultado esperado verificável; se algum falhar, pare e corrija antes de divulgar o link.

VERIFICAÇÃO	COMO	ESPERADO
Página pública	abrir `/` em janela anônima	carrega sem login, com comarcas no mapa e na lista
Saúde do serviço	`/healthz` responde ao health check (200 sem tocar no banco)	status 200 no painel do Render
Admin bloqueado	abrir `/admin` sem login	não mostra gestão; oferece entrar com Google
Login Google	entrar com a conta da allowlist	volta autenticado e o menu mostra o papel de admin
Login de terceiro	entrar com outra conta	entra como comum e só vê o que foi liberado
Pedido de acesso	pedir acesso a uma comarca	pedido registrado; comportamento conforme decisão 2B
Edição	editar um contato como admin	alteração salva e visível após recarregar
Celular	abrir em 390 px de largura	sem rolagem horizontal e menu utilizável
HTTPS	abrir `http://`	redireciona para `https://`

Primeira visita demora: no plano free o serviço hiberna após alguns minutos sem acesso e a primeira requisição pode levar até ~50 s. Se for apresentar o link a um recrutador, abra a página alguns minutos antes para acordar o serviço.

06 Se algo der errado

`redirect_uri_mismatch`

URL ou caminho do retorno diferente do registrado. Confira esquema (`https`), domínio exato e `/oauth/callback`.

Admin sem permissão

O e-mail do login não está em `ADMIN_EMAILS`, ou a variável foi alterada mas o serviço ainda não reiniciou.

Erro 502 no primeiro acesso

Serviço hibernando ou reiniciando. Recarregar após o `Live`; se persistir, ver o log de boot.

Dados sumiram

Esperado no plano free: o banco foi recriado a partir do seed. Para persistir, seria preciso disco pago ou banco externo.

Voltar atrás

- **Rollback de deploy:** no Render, abrir o deploy anterior em **Events** e usar **Rollback to this deploy**.
- **Desligar a vitrine: Settings → Suspend** mantém a configuração e tira o site do ar.
- **Trocar o OAuth:** revogar o cliente no Google Cloud e gerar outro JSON; atualizar a variável no Render.

07 Registro do deploy

PREENCHER

Data e hora	.....
Commit publicado	.....
URL pública	.....
Admin da allowlist	.....
Aprovação de acesso (0 = manual)	.....
SMTP configurado	.....
Checklist da seção 05	.....
Observações	....

08 Anexo — variáveis de ambiente

VARIÁVEL	ONDE VIVE	OBRIGATÓRIA	FUNÇÃO
`ADMIN_EMAILS`	painel Render	sim	allowlist de administradores
`APP_BASE_URL`	painel Render	sim	URL pública usada em links e no login
`GOOGLE_CLIENT_JSON`	painel Render (segredo)	sim	credencial OAuth tipo web
`LOCAL_LOGIN`	render.yaml	sim (0)	desliga usuário/senha local em produção
`INTRANET_ONLY`	render.yaml	sim (0)	0 libera acesso fora da rede local
`SIMULACAO_AUTO_APROVAR`	render.yaml	sim (0 — somente leitura)	1 aprova pedidos na hora; 0 exige aprovação do admin
`PYTHON_VERSION`	render.yaml	sim	fixa a versão do Python do runtime
`NOTIFY_EMAILS`	painel Render	não	destinatários de avisos internos
`GOOGLE_TEST_EMAILS`	painel Render	não	contas liberadas em modo de teste local
`SMTP_HOST · PORT · USER · PASS · FROM`	painel Render (segredo)	não	envio de e-mail de aprovação e recusa

Regra de ouro: segredo nunca entra no repositório. O que está versionado é apenas o desenho do serviço; credencial vive no painel.